

EMBEDDED SYSTEMS

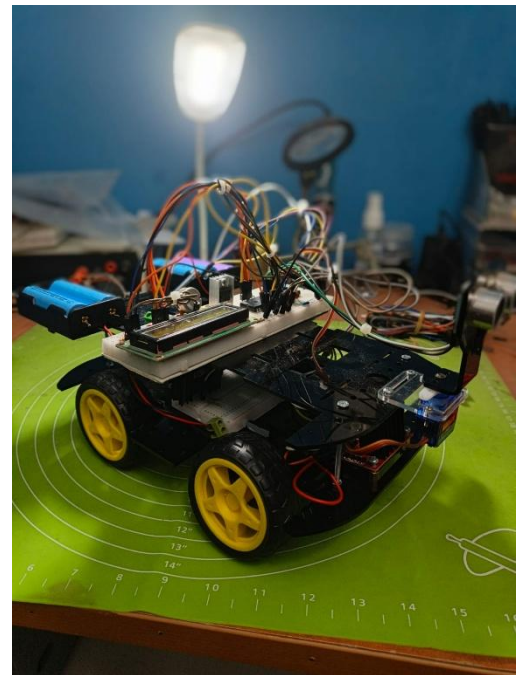


Thebes Institute
for Engineering

Multi-Mode ATmega32A Car Project

Students GROUP:

- **Basel Mohamed Mostafa**
- **Ziad Refaie Abu AlFtooh**
- **Hussin Hesham Ali**
- **Andrew Emad Sadek**
- **Mohamed Ahmed Abdelnour**
- **Abdullah Sayed Mohamed**
- **Hassan Elkenany**
- **Fatma Ahmed Elawady**
- **Hagar Mohamed Abdelrahman**
- **Mariam Nashaat Boshra**
- **Mostafa Ayman**



ABSTRACT

This report details the design and implementation of a custom robotic vehicle powered by an ATmega32A microcontroller. The system features a Finite State Machine (FSM) architecture that allows seamless, real-time transition between two distinct operating states: Manual Bluetooth Control and Autonomous Obstacle Avoidance. By developing a complete software stack from scratch, including the Microcontroller Abstraction Layer (MCAL) and Hardware Abstraction Layer (HAL)—the project successfully demonstrates non-blocking peripheral management, hardware PWM signal generation, and environmental spatial awareness.

CONTENTS

Chapter 1: INTRODUCTION.....	3
1.1 Overview:	3
1.2 Scope of the Project.....	3
Chapter 2:.....	4
2.1 The Microcontroller	4
2.2 Actuators & Motor Control	4
2.3 Sensors & Communication	4
2.4 Power Management.....	4
Chapter 3: Software & Architecture	6
3.1 Layered Architecture Overview	6
3.2 Peripheral Utilization	6
3.3 The Finite State Machine (FSM).....	6
Chapter 4: System Operating Modes.....	8
4.1 Mode 0: Manual Bluetooth Control	8
4.2 Mode 1: Autonomous Obstacle Avoidance.....	8
Chapter 6: Conclusion	9
6.1 Conclusion	9

CHAPTER 1: INTRODUCTION

1.1 OVERVIEW:

Robotic vehicles are frequently deployed in environments that are either inaccessible or hazardous to humans. Depending on the scenario, these robots must either be teleoperated by a human driver or possess the intelligence to navigate independently. This project bridges both paradigms by creating a unified robotic platform capable of remote exploration and autonomous environmental reaction.

The primary objective of this project was to engineer a reliable embedded system capable of:

- Processing non-blocking wireless commands via Bluetooth.
- Generating precise pulse-width modulation (PWM) signals for DC motor speed control and servo positioning.
- Utilizing ultrasonic time-of-flight data to map immediate surroundings and avoid collisions.
- Providing real-time system feedback via an LCD interface.

1.2 SCOPE OF THE PROJECT

The project encompasses both hardware and software development. Hardware tasks included chassis assembly, power distribution, and component wiring. Software tasks involved writing pure C drivers for the ATmega32A registers, establishing layered software architecture, and writing the application-level FSM logic to safely manage state transitions.

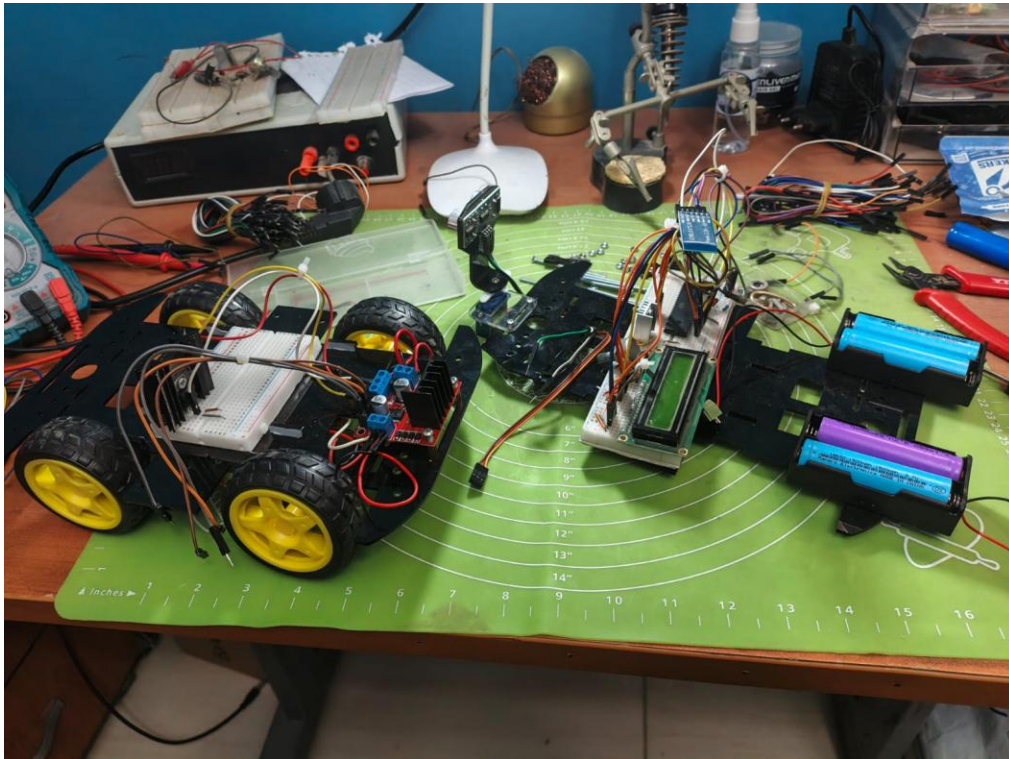


Figure 1: Building the Chassis Car.

CHAPTER 2:

2.1 THE MICROCONTROLLER

The brain of the system is the **ATmega32A** 8-bit AVR microcontroller, clocked externally at 16MHz. This MCU was selected for its robust suite of hardware timers, standard communication protocols (USART), and sufficient GPIO pins to handle multiple actuators and sensors simultaneously.

2.2 ACTUATORS & MOTOR CONTROL

- **DC Motors & L298N Driver:** The vehicle utilizes a four-wheel skid-steering drive system. Power is routed through an L298N Dual H-Bridge motor driver, allowing the MCU to control both the direction and speed of the left and right motor banks.
- **SG-90 Micro Servo:** Mounted at the front of the vehicle, this servo acts as a panning "neck" for the ultrasonic sensor, allowing the robot to scan its environment left and right.

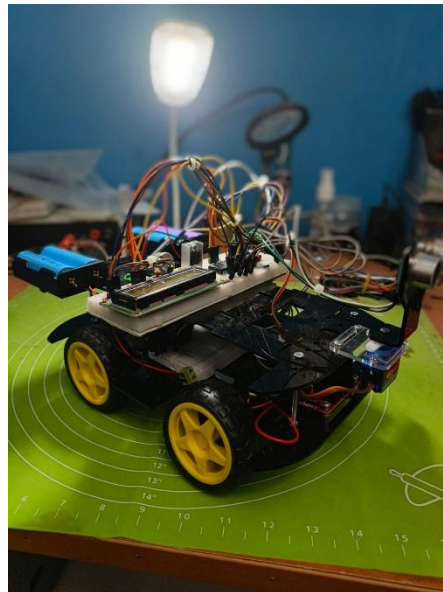


Figure 2: Finished Car structure.

2.3 SENSORS & COMMUNICATION

- **HC-05 Bluetooth Module:** Connected to the MCU's hardware UART pins, this module receives serial commands from an Android controller application at a baud rate of 9600.
- **HC-SR04 Ultrasonic Sensor:** This acoustic sensor acts as the robot's "eyes," calculating the distance to nearby obstacles by measuring the time-of-flight of high-frequency sound pulses.

2.4 POWER MANAGEMENT

The system is powered by a high-capacity dual-18650 lithium-ion battery pack (providing ~7.4V to 8.4V). Power distribution is carefully managed to isolate the sensitive MCU logic from the high-current spikes

generated by the DC motors and the SG-90 servo, utilizing the L298N's onboard 5V regulator for logic-level components.

CHAPTER 3: SOFTWARE & ARCHITECTURE

3.1 LAYERED ARCHITECTURE OVERVIEW

To ensure code reusability and maintainability, the software was strictly divided into layers:

1. **MCAL (Microcontroller Abstraction Layer):** Custom drivers for DIO, TIMER, and USART peripherals.
2. **HAL (Hardware Abstraction Layer):** Component-specific drivers for the LCD, L298N Motors, HC-SR04, and SG-90.
3. **Application Layer:** The FSM and primary logic loops executing in main.c.

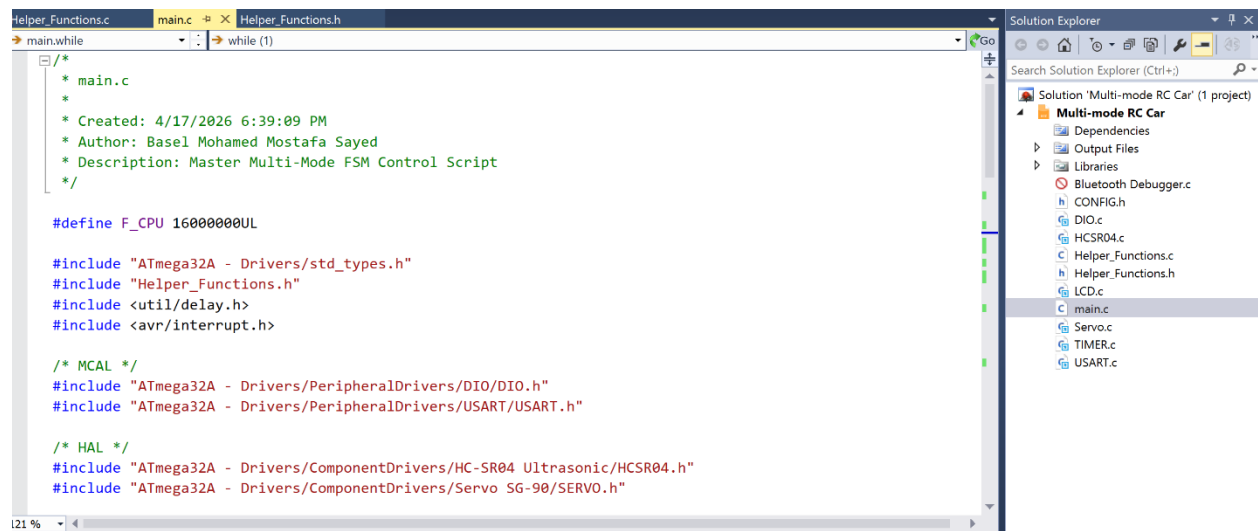


Figure 3: Main.c Code start.

3.2 PERIPHERAL UTILIZATION

Efficient resource management was critical for multi-mode operation:

- **Timer 0:** Configured in Fast PWM mode with a prescaler of 64 to generate the duty cycles required for dynamic DC motor speed control.
- **Timer 2:** Configured as an 8-bit Fast PWM generator to output a ~61Hz control signal for the SG-90 servo, freeing up the 16-bit Timer 1 for other high-precision tasks.
- **USART (Interrupt-Driven):** Configured to receive data asynchronously (USART_receiveByteNonBlocking). This ensures the CPU is never stalled waiting for user input, which is critical for the FSM's stability.

3.3 THE FINITE STATE MACHINE (FSM)

The core architecture of main.c relies on an FSM governed by a systemMode variable. By utilizing a non-blocking listener loop, the MCU can instantly toggle between MODE_MANUAL and MODE_AUTONOMOUS without relying on dangerous software delays.

```
Helper_Functions.c  main.c  Helper_Functions.h
main.while  while (1)
#include "ATmega32A - Drivers/ComponentDrivers/Servo SG-90/SERVO.h"

int main(void) {
    /* Define Variables */
    u8 receivedCommand = 0;
    u8 systemMode = MODE_MANUAL;

    // 1. Wake up and configure all peripherals, timers, pins, and interrupts
    initializeProgram();

    // 2. Clear the LCD and set the initial standby state
    LCD_clear();
    Robot_UpdateLCD(systemMode, 0, "Standby");

    while (1) {

        // =====
        // BLUETOOTH MOOD NON-BLOCKING (KEY '1' to turn on Bluetooth mode)
        // =====
        if (USART_receiveByteNonBlocking(&receivedCommand) == F_OK) {

            // --- Switching Logic ---
            if (receivedCommand == '1' && systemMode != MODE_MANUAL) {
                systemMode = MODE_MANUAL;
                Motors_off(); // Turn off all motors during transition
                SERVO_setAngle_Timer2(90); // Center the servo head
                LCD_clear();
                Robot_UpdateLCD(systemMode, 0, "Standby");
                USART_sendString((const u8*)"[SYS] Switched to MANUAL Mode\r\n");
            }
        }
    }
}
```

```
Helper_Functions.c  main.c  Helper_Functions.h
main.while  while (1)
    Robot_UpdateLCD(systemMode, 0, "Standby");
    USART_sendString((const u8*)"[SYS] Switched to MANUAL Mode\r\n");
}
else if (receivedCommand == '2' && systemMode != MODE_AUTONOMOUS) {
    systemMode = MODE_AUTONOMOUS;
    Motors_off(); // Turn off all motors during transition
    LCD_clear();
    Robot_UpdateLCD(systemMode, 0, "Starting...");
    USART_sendString((const u8*)"[SYS] Switched to AUTONOMOUS Mode\r\n");
}
// --- MANUAL DRIVING LOGIC ---
else if (systemMode == MODE_MANUAL) {
    // Pass driving characters (F, B, R, L, S) to the motor parser
    ExecuteBluetoothMotorCommand(receivedCommand);
}
// =====
// THE AUTONOMOUS MODE (KEY '2' Using Terminal to activate)
// =====
if (systemMode == MODE_AUTONOMOUS) {
    // Hand control over to the ultrasonic sensor and servo FSM
    Autonomous_Routine();
}
}
return 0;
}
```

```
Helper_Functions.c  main.c  Helper_Functions.h
Helper_Functions.h  G:\Work Projects\Embedded - IoT Projects\Multi-mode ATmega32A Car\Multi-mode RC Car\Helper_Functions.h
/*
 *
 * Start the program, calling any initialization of peripherals
 * void initializeProgram(void);
 *
 * =====
 * Motor Control Functions
 * =====
 *
 * Function to only enable the left motors (steer the Vehicle left)
 * void Motors_left(u8 copy_u8Speed);
 *
 * Function to only enable the right motors (steer the vehicle right)
 * void Motors_right(u8 copy_u8Speed);
 *
 * This function turns on all motors connected to go forward
 * void Motors_forward(u8 copy_u8Speed);
 *
 * This function turns on all motors connected to go backwards
 * void Motors_reverse(u8 copy_u8Speed);
 *
 * Function to turn off all motors (standby)
 * void Motors_off(void);
 *
 */
```

CHAPTER 4: SYSTEM OPERATING MODES

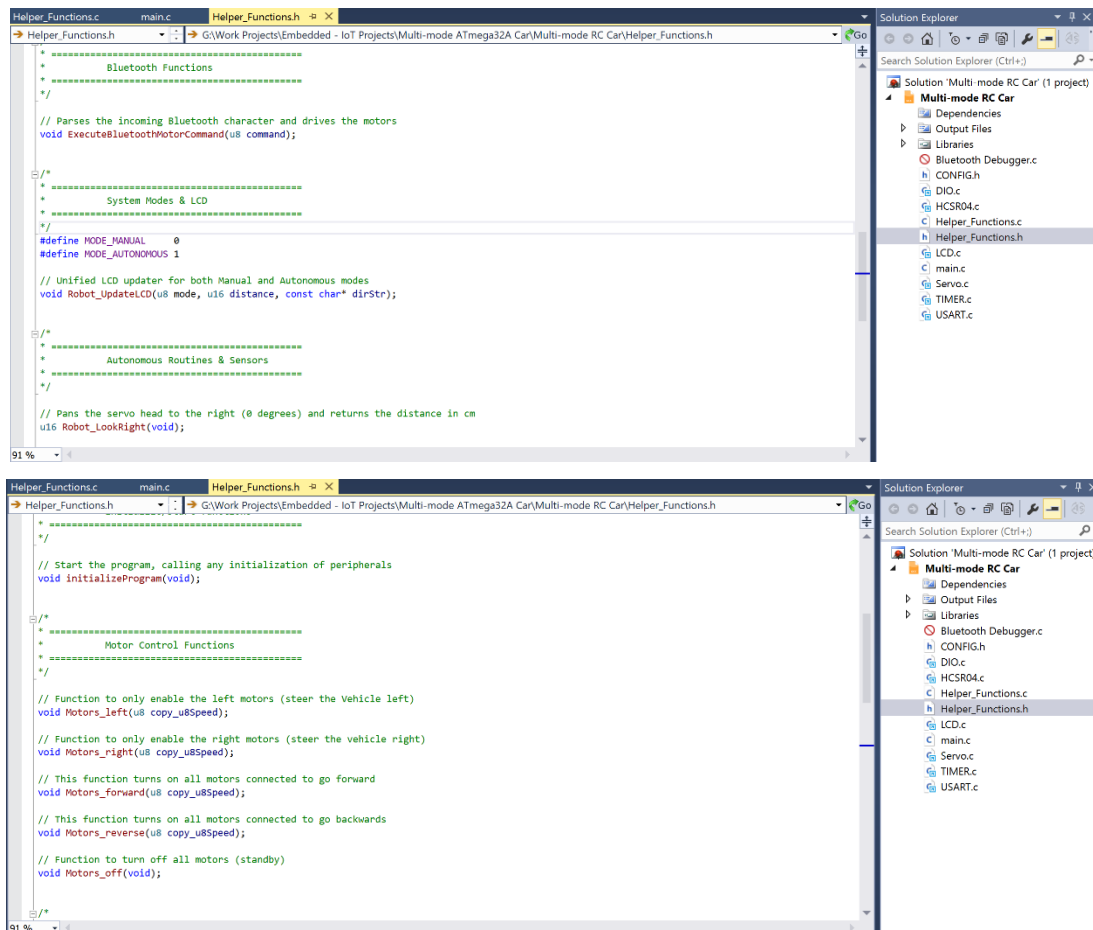
4.1 MODE 0: MANUAL BLUETOOTH CONTROL

In this state, the robot acts as a standard teleoperated vehicle. The USART peripheral parses ASCII characters ('F', 'B', 'L', 'R', 'S') sent from the smartphone application. The application layer routes these commands to the Motor HAL to execute forward, reverse, and pivot-turn maneuvers. The 16x2 LCD is continuously updated to display the current connection status and active movement command.

4.2 MODE 1: AUTONOMOUS OBSTACLE AVOIDANCE

Upon receiving the override command ('2'), the FSM shifts into autonomous operation.

1. **Cruising:** The robot drives forward continuously while pinging the HC-SR04.
2. **Detection & Braking:** If an object is detected within 25cm, the robot immediately halts, reverses for 400ms to clear physical space, and stops.
3. **Scanning:** The SG-90 servo sweeps the sensor to 0° (Right) and 180° (Left), taking distance readings at each vector.
4. **Decision Making:** The MCU compares the left and right distance variables and steers the motors toward the most open path. If both paths are blocked, a 180° U-turn failsafe is executed.



CHAPTER 6: CONCLUSION

6.1 CONCLUSION

The project successfully met all design requirements. By developing custom, layered C drivers rather than relying on pre-built Arduino libraries, the project demonstrates a deep understanding of AVR register manipulation. The non-blocking FSM allows for a robust, crash-free user experience, proving that a single 8-bit microcontroller can simultaneously handle wireless communications, LCD updates, mechanical sweeping, and spatial mapping.

Thank♥you